

Nomlangan obyektlarni tanib olish (NER)

Tabiiy tilni qayta ishlash — 10-ma'ruza

A.A. Abdulali

11:00–12:20 · mavzu №10

30-iyun 2026

Prezentatsiya rejasi

- 1 Kirish: matndan obyektlarni ajratamiz
- 2 NER vazifasi va teglash sxemalari
- 3 LSTM asosidagi NER arxitekturasini
- 4 Bi-LSTM bilan kontekstni yaxshilash
- 5 NER samaradorligini baholash
- 6 Xulosa va amaliyotga ko'prik

Takrorlash: o'tgan darsdan ikki savol

1-savol

L9 da **Bi-LSTM** matn generatsiya qila olmasligini ko'rdik. Nega? (Maslahat: u nimanani ko'rishi kerak edi?)

2-savol

Unda butun gapni (chap **va** o'ng kontekst) ko'rish qaysi vazifada **foydali**?

Takrorlash: o'tgan darsdan ikki savol

1-savol

L9 da **Bi-LSTM** matn generatsiya qila olmasligini ko'rdik. Nega? (Maslahat: u nimanani ko'rishi kerak edi?)

2-savol

Unda butun gapni (chap **va** o'ng kontekst) ko'rish qaysi vazifada **foydali**?

1-javob

Generatsiyada **kelajak** hali yo'q; Bi-LSTM esa o'ng (kelajak) kontekstni talab qiladi $\Rightarrow$ generatsiyaga yaramaydi.

2-javob

Teglash (ketma-ketlik labellash) — bugungi mavzu **NER**. Butun gap ma'lum, har token uchun teg beramiz.

Siz bugungi dars oxirida quyidagilarni bajara olasiz:

- ❶ **Tushuntira olasiz:** NER vazifasi va IOB2 teglash sxemasini (B-/I-/O).
- ❷ **Qo'llay olasiz:** IOB2 ni qo'lda — "Malika Toshkentda ishlaydi" → [B-PER, B-LOC, O, O].
- ❸ **Tushuntira olasiz:** LSTM/Bi-LSTM NER arxitekturasini (har token uchun teg).
- ❹ **Hisoblay olasiz:** entity-darajali precision/recall/F1 ni.

L7–L9 dan: LSTM, Bi-LSTM. L2 dan: precision/recall/F1. Matematikadan: nisbat, o'rtacha (harmonik).

Bugungi dars rejasi:

Tsikl	Mavzu	Vaqt	Hosil natija	bo'luvchi
1	NER vazifasi va teglash sxemalari	17 daq	[B-PER, B-LOC, 0, 0]	
2	LSTM asosidagi NER arxitekturasini	16 daq	har token uchun teg	
3	Bi-LSTM bilan kontekstni yaxshilash	16 daq	chap + o'ng kontekst	
4	NER samaradorligini baholash	17 daq	entity F1 = 0.5	
	Xulosa va 10-amaliyotga ko'rsatmalar	10 daq	Ertaga nima qurishingiz	

Bu dars Bi-LSTM ning asl uyiga olib keladi

L9 da Bi-LSTM generatsiyaga yaramadi; bugun u **ketma-ketlik teglash**da porlaydi. Bugungi [B-PER, B-LOC, 0, 0] ertangi P10 da assert bo'ladi.

Bugungi dars uchun muammo: matndagi nomlarni qanday topamiz?

Sodda yechim qayerda qiynaladi:

Lug'at qidiruvi yetarli emas

Tayyor ismlar ro'yxati **yangi** nomlarni topa olmaydi va kontekstni ko'rmaydi (Olma — ism yoki meva?).

Qo'shimchalar chegarani buzadi

Toshkentda — joy nomi + "-da". Naive bo'lakka ajratish entity chegarasini noto'g'ri belgilaydi.

Bugungi yechim:

- 1 NER ni **ketma-ketlik teglash** deb qaraymiz (har token — teg).
- 2 **IOB2** sxemasi entity chegarasini aniq belgilaydi.
- 3 **Bi-LSTM** chap va o'ng kontekstdan teg uchun foydalanadi.

Natija: kontekstga sezgir, yangi nomlarni ham topadigan tizim.

Prezentatsiya rejasi

- 1 Kirish: matndan obyektlarni ajratamiz
- 2 NER vazifasi va teglash sxemalari**
- 3 LSTM asosidagi NER arxitekturasini
- 4 Bi-LSTM bilan kontekstni yaxshilash
- 5 NER samaradorligini baholash
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: har so'zga "rol" yorlig'i yopishtiramiz

NER — matndagi **nomlangan obyektlarni** topish va turini aniqlash:

- **PER** — shaxs (Malika, Akmal)
- **LOC** — joy (Toshkent, Samarqand)
- **ORG** — tashkilot (BMT, universitet)

Har **token**ga yorliq beramiz — bu ketma-ketlik teglash.

Muammo: entity bir **nechta** so'zdan iborat bo'lishi mumkin ("Toshkent shahri"). Boshini va davomini qanday ajratamiz?
⇒ IOB2 sxemasi.

IOB2 teglash sxemasi: rasmiy ta'rif

- B-XXX — entity **boshi** (Begin), turi XXX.
- I-XXX — entity **ichi/davomi** (Inside), turi XXX.
- O — entity **emas** (Outside).

bunda: XXX — entity turi (PER/LOC/ORG); IOB2 da **har** entity B- bilan boshlanadi (IOB1 dan farqi); I- faqat B- dan keyin keladi. **Misol:** "Toshkent shahri" → B-LOC I-LOC (ikki tokenli bitta joy).

Keltirib chiqarish: nega oddiy PER/LOC/O yetarli emas?

1-qadam. Faqat tur yorlig'i (PER/LOC/O) bilan ketma-ket ikki entity **birlashib** ketadi:
"Malika Akmal" → PER PER — bir entitymi yoki ikkitami?

Keltirib chiqarish: nega oddiy PER/LOC/O yetarli emas?

1-qadam. Faqat tur yorlig'i (PER/LOC/O) bilan ketma-ket ikki entity **birlashib** ketadi: "Malika Akmal" → PER PER — bir entitymi yoki ikkitami? **2-qadam.** B-/I- ajratuvi bu noaniqlikni yo'qotadi: B-PER B-PER (ikki shaxs) vs B-PER I-PER (bir to'liq ism).

Keltirib chiqarish: nega oddiy PER/LOC/O yetarli emas?

1-qadam. Faqat tur yorlig'i (PER/LOC/O) bilan ketma-ket ikki entity **birlashib** ketadi: "Malika Akmal" → PER PER — bir entitymi yoki ikkitami? **2-qadam.** B-/I- ajratuvi bu noaniqlikni yo'qotadi: B-PER B-PER (ikki shaxs) vs B-PER I-PER (bir to'liq ism). **3-qadam.** Demak chegara ma'lumoti yorliqqa **kiritiladi**:

$$\text{token}_i \rightarrow \{B-, I-\}\text{-tur yoki } 0$$

Xulosa: IOB2 — entity **chegarasini** ham, **turini** ham bitta yorliqda kodlaydi.

Qo'lda misol: IOB2 teglash — [B-PER, B-LOC, 0, 0]

Tokenlar: [Malika, Toshkentda, ishlaydi, .]

Token	Tahlil	Teg
Malika	shaxs ismi (entity boshi)	B-PER
Toshkentda	joy nomi + "-da" suffiks (boshi)	B-LOC
ishlaydi	fe'l, entity emas	0
.	tinish belgisi, entity emas	0

Toshkentda = B-LOC: "-da" suffiks entity **chegarasini o'zgartirmaydi** (bu o'zbek uchun muhim — [M]).

Ertangi P10 birinchi assert:
tags == ["B-PER", "B-LOC",
"0", "0"]

Ma'ruza L10 [I1]-slayd

Sizning vazifangiz: ikki tokenli entity

Savol — 90 soniya

Gap: "Akmal Navoiy shahrida yashaydi". Tokenlar: [Akmal, Navoiy, shahrida, yashaydi].

IOB2 teglari? (Maslahat: "Navoiy shahrida" — bir joy, ikki token.)

Sizning vazifangiz: ikki tokenli entity

Savol — 90 soniya

Gap: "Akmal Navoiy shahrida yashaydi". Tokenlar: [Akmal, Navoiy, shahrida, yashaydi].

IOB2 teglari? (Maslahat: "Navoiy shahrida" — bir joy, ikki token.)

Javob

[B-PER, B-LOC, I-LOC, 0]

Akmal=B-PER; Navoiy=B-LOC (joy boshi); shahrida=I-LOC (joyning davomi); yashaydi=0. Ikki tokenli entity B- + I- bilan belgilanadi.

Kod ↔ tushuncha: IOB2 teglar

```
# teglangan jumla: (token, teg) juftlari
sentence = [
    ("Malika", "B-PER"),
    ("Toshkentda", "B-LOC"),
    ("ishlaydi", "O"),
    (".", "O"),
]
tags = [tag for _, tag in sentence]
print(tags)      # ['B-PER', 'B-LOC', 'O', 'O']

TAGSET = ["O", "B-PER", "I-PER",
          "B-LOC", "I-LOC", "B-ORG", "I-ORG"]
```

Kod → tushuncha:

Kod	Ma'no
B-PER	shaxs boshi
I-LOC	joy davomi
O	entity emas

P10 da m10 NERTagger aynan shunday (token, teg) juftlarida o'qiydi.

Teglash tuzog'i: noto'g'ri teg ketma-ketligi

Imkonsiz ketma-ketlik

0 dan keyin to'g'ridan-to'g'ri I-LOC **kelishi mumkin emas** (I- faqat B- dan keyin). Model ba'zan shunday xato chiqaradi.

Yechim: chiqishda **CRF** qatlami (teglar orasidagi o'tishlarni o'rganadi) yoki post-protsessing bilan tuzatish (P10 da ko'ramiz).

Prezentatsiya rejasi

- 1 Kirish: matndan obyektlarni ajratamiz
- 2 NER vazifasi va teglash sxemalari
- 3 LSTM asosidagi NER arxitekturasini**
- 4 Bi-LSTM bilan kontekstni yaxshilash
- 5 NER samaradorligini baholash
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: har token uchun yorliq bashorat qilamiz

Tasniflash (L7–L8) butun gapga **bitta** yorliq berdi.

NER — har **token**ga yorliq (ketma-ket teglash):

token → embedding → LSTM → har qadam yashirin
holatdan **teg taqsimoti** (softmax).

“ko’p-dan-ko’p” (L9 generatsiyaga o’xshash), lekin chiqish — so’z emas, **teg**. Va Bi-LSTM mumkin (kelajak ma’lum).

$$h_t = \text{LSTM}(x_t, h_{t-1}), \quad \hat{y}_t = \text{softmax}(W h_t + b)$$

bunda: x_t — t token embeddingi; h_t — yashirin holat; W — teg-chiqish og'irligi; $\hat{y}_t$ — t token uchun teglar ustidan taqsimot; tanlangan teg = $\arg \max \hat{y}_t$. Har **qadamda** chiqish bor (L9 generatsiyadagi kabi), lekin teglar ustidan ($|\text{TAGSET}|$ ta sinf), so'zlar ustidan emas.

1-qadam. LSTM har token uchun h_t beradi — bu tokenning kontekstli ifodasi.

Keltirib chiqarish: token holatidan tegga

1-qadam. LSTM har token uchun h_t beradi — bu tokenning kontekstli ifodasi. **2-qadam.** h_t ni teglar soniga ($|\text{TAGSET}|$) proyeksiya qilamiz: $z_t = Wh_t + b$.

Keltirib chiqarish: token holatidan tegga

1-qadam. LSTM har token uchun h_t beradi — bu tokenning kontekstli ifodasi. **2-qadam.** h_t ni teglar soniga ($|\text{TAGSET}|$) proyeksiya qilamiz: $z_t = Wh_t + b$. **3-qadam.** Softmax bilan teg taqsimotiga aylantiramiz va eng ehtimoliysini olamiz:

$$\hat{y}_t = \text{softmax}(z_t), \quad \text{teg}_t = \arg \max \hat{y}_t$$

Xulosa: bu — L6 dagi softmax tasnifi, lekin **har token** uchun takrorlanadi.

Qo'lda misol: bitta token uchun teg tanlash

TAGSET = [0, B-PER, B-LOC]; ballar $z = [0.0, 2.0, 1.0]$.

Softmax va tanlov:

$$\begin{aligned}\hat{y} &= \frac{[e^0, e^2, e^1]}{e^0 + e^2 + e^1} \\ &= \frac{[1.0, 7.39, 2.72]}{11.11} = [0.090, 0.665, 0.245]\end{aligned}$$

$\arg \max \Rightarrow \text{B-PER (0.665)}.$

Model Malika ni B-PER deb tegladi — to'g'ri. Har token uchun shu qadam takrorlanadi.

Sizning vazifangiz: boshqa token uchun teg

Savol — 60 soniya

TAGSET = [0, B-PER, B-LOC], token "ishlaydi" uchun ballar $z = [3.0, 1.0, 0.0]$:

qaysi teg tanlanadi?

Sizning vazifangiz: boshqa token uchun teg

Savol — 60 soniya

TAGSET = [0, B-PER, B-LOC], token "ishlaydi" uchun ballar $z = [3.0, 1.0, 0.0]$:

qaysi teg tanlanadi?

Javob

Eng katta ball $z_1 = 3.0$ ($\arg \max$) $\Rightarrow 0$.

Softmax tartibni saqlaydi, shuning uchun eng katta ballli teg tanlanadi. *ishlaydi* fe'l $\Rightarrow 0$ (entity emas)
— to'g'ri.

Kod $\leftrightarrow$ formula: per-token teglash

```
1 import torch.nn as nn
2
3 class NERModel(nn.Module):
4     def __init__(self, vocab, dim, hid, n_tags):
5         super().__init__()
6         self.emb = nn.Embedding(vocab, dim)
7         self.lstm = nn.LSTM(dim, hid, batch_first=True)
8         self.fc = nn.Linear(hid, n_tags)
9
10    def forward(self, x):
11        e = self.emb(x)
12        out, _ = self.lstm(e)          # HAR token
13                                       # uchun  $h_t$ 
14        return self.fc(out)          #  $(B, T,$ 
15                                        $n\_tags)$ 
```

Kod $\rightarrow$ formula:

Kod	Formula
lstm(e)	h_t (har token)
fc(out)	$Wh_t + b$
(B,T,n_tags) per-token $\hat{y}_t$	

Tasniflashdan farqi: out (har token), h_T (oxirgi) emas. P10 da m10 shuni quradi.

Arxitektura tuzog'i: bir tomonlama kontekst yetishmaydi

O'ng kontekst kerak

Bir tomonlama LSTM "Olma" ni teglaganda faqat **chap**ni ko'radi. "Olma keldi" (ism) vs "Olma yedim" (meva) — farqni o'ng kontekst hal qiladi.

Yechim: Bi-LSTM — chap **va** o'ng kontekstni birlashtiradi (3-tsikl). NER da bu deyarli har doim yordam beradi.

Prezentatsiya rejasi

- 1 Kirish: matndan obyektlarni ajratamiz
- 2 NER vazifasi va teglash sxemalari
- 3 LSTM asosidagi NER arxitekturas
- 4 Bi-LSTM bilan kontekstni yaxshilash**
- 5 NER samaradorligini baholash
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: teglash uchun butun gap ko'rinadi

L9 dagi Bi-LSTM bu yerda **ideal**: NER da butun gap oldindan ma'lum (generatsiya emas), shuning uchun o'ng kontekstni ham ishlatish mumkin.

Har token uchun: $h_t = [\vec{h}_t; \overleftarrow{h}_t]$ — chap va o'ng kontekst birga.

“Toshkentda” ni teglaganda model oldingi ("Malika") va keyingi ("ishlaydi") so'zlarni ham ko'radi $\Rightarrow$ aniqroq teg.

$$h_t = [\vec{h}_t; \overleftarrow{h}_t], \quad \hat{y}_t = \text{softmax}(W h_t + b)$$

bunda: $\vec{h}_t$ — chapdan (oldingi tokenlar); $\overleftarrow{h}_t$ — o'ngdan (keyingi tokenlar); $[\ ; \]$ — birlashtirish; h_t o'lchami $2 \times$; W shunga mos kengaytiriladi. Har token endi **to'liq** kontekstga ega — NER aniqligi sezilarli oshadi (Lample va boshq., 2016).

Keltirib chiqarish: nega NER da generatsiyadan farqli ishlaydi?

1-qadam. Generatsiyada (L9) t qadamda **kelajak** ($x_{t+1} \dots$) hali yaratilmagan $\Rightarrow \overleftarrow{h}_t$ ni hisoblab bo'lmaydi.

Keltirib chiqarish: nega NER da generatsiyadan farqli ishlaydi?

1-qadam. Generatsiyada (L9) t qadamda **kelajak** ($x_{t+1} \dots$) hali yaratilmagan $\Rightarrow \overleftarrow{h}_t$ ni hisoblab bo'lmaydi. **2-qadam.** Teglashda (NER) butun gap **oldindan** kiritiladi $\Rightarrow \overleftarrow{h}_t$ ni erkin hisoblaymiz.

Keltirib chiqarish: nega NER da generatsiyadan farqli ishlaydi?

1-qadam. Generatsiyada (L9) t qadamda **kelajak** ($x_{t+1} \dots$) hali yaratilmagan $\Rightarrow \overleftarrow{h}_t$ ni hisoblab bo'lmaydi. **2-qadam.** Teglashda (NER) butun gap **oldindan** kiritiladi $\Rightarrow \overleftarrow{h}_t$ ni erkin hisoblaymiz. **3-qadam.** Shuning uchun teglashda chap va o'ng birlashadi:

$$h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t]$$

Xulosa: sababiyat (causality) cheklovi yo'q $\Rightarrow$ Bi-LSTM teglash uchun — aynan to'g'ri vosita.

Qo'lda misol: chap va o'ng kontekstni birlashtiramiz

$\vec{h} = [0.2, 0.8]$ (chap: gap boshi), $\overleftarrow{h} = [0.9, 0.1]$ (o'ng: "...shahriga bordi").

Birlashtiramiz:

$$\begin{aligned} h_t &= [\vec{h} ; \overleftarrow{h}] \\ &= [0.2, 0.8, 0.9, 0.1] \end{aligned}$$

O'ng kontekst ("shahriga") Olma ni **joy** ekanini ko'rsatadi $\Rightarrow$ B-LOC. Bir tomonlama buni topa olmasdi.

O'lcham $2 \rightarrow 4$; endi teg uchun to'liq kontekst bor.

Sizning vazifangiz: birlashtirilgan holat o'lchami

Savol — 60 soniya

Agar har yo'nalish holati 64 o'lchamli bo'lsa (Bi-LSTM, hidden= 64):

birlashtirilgan h_t o'lchami nechta?

Sizning vazifangiz: birlashtirilgan holat o'lchami

Savol — 60 soniya

Agar har yo'nalish holati 64 o'lchamli bo'lsa (Bi-LSTM, hidden= 64):

birlashtirilgan h_t o'lchami nechta?

Javob

$64 + 64 = \mathbf{128}$ ($2 \times \text{hidden}$).

Shuning uchun chiqish qatlami W kirishi 128 bo'ladi ($\vec{h}$ va $\overleftarrow{h}$ birlashgan).

Kod ↔ formula: Bi-LSTM NER

```
import torch.nn as nn

class BiLSTMNER(nn.Module):
    def __init__(self, vocab, dim, hid, n_tags):
        super().__init__()
        self.emb = nn.Embedding(vocab, dim)
        self.lstm = nn.LSTM(dim, hid, batch_first=True,
                             bidirectional=True)
        self.fc = nn.Linear(2 * hid, n_tags)  # 2*hid!

    def forward(self, x):
        out, _ = self.lstm(self.emb(x))  # (B, T, 2*hid)
        return self.fc(out)
```

Kod → formula:

Kod	Formula
bidirectional	$\vec{h}, \overleftarrow{h}$
$2 * hid$	[;] o'lcham
fc(out)	per-token teg

2*hid — eng ko'p unutiladigan joy. P10 da m10 shu arxitekturani quradi.

Bi-LSTM tuzog'i: chiqish o'lchamini unutish

Eng tez-tez xato

`bidirectional=True` qilib, chiqish qatlami kirishini `hid` qoldirish $\Rightarrow$ o'lcham mos kelmaydi (xato). Kirish $2*hid$ bo'lishi shart.

Yodda tuting: `bidirectional` $\Rightarrow$ chiqish o'lchami ikki baravar. Bu L9 dagi nusxaning amaliy oqibati.

Prezentatsiya rejasi

- 1 Kirish: matndan obyektlarni ajratamiz
- 2 NER vazifasi va teglash sxemalari
- 3 LSTM asosidagi NER arxitekturasini
- 4 Bi-LSTM bilan kontekstni yaxshilash
- 5 NER samaradorligini baholash**
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: to'g'ri entity — chegara VA tur to'g'ri

NER da baholash **entity-darajali**: bir entity to'g'ri sanaladi faqat **ham chegarasi, ham turi** to'g'ri bo'lsa.

Token-darajali aniqlik chalg'itadi: ko'p 0 teglari bor, ularni to'g'ri topish oson $\Rightarrow$ sun'iy yuqori ball.

Shuning uchun sequeval entity-darajali precision/recall/F1 hisoblaydi — bu NER ning standart o'lchovi.

Entity-darajali F1: rasmiy ta'rif

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}$$

bunda: TP — to'g'ri topilgan entitylar (chegara + tur mos); FP — noto'g'ri taklif qilingan; FN — o'tkazib yuborilgan haqiqiy entitylar. Entity — to'liq span, token emas. Token-darajali emas, **entity-darajali** — qisman mos (chegarasi to'g'ri, turi xato) ham **xato** sanaladi.

1-qadam. Har bashorat entity ni haqiqiy (gold) entitylar bilan solishtiramiz — span **va** tur bir xilmi?

Keltirib chiqarish: TP/FP/FN ni sanaymiz

1-qadam. Har bashorat entity ni haqiqiy (gold) entitylar bilan solishtiramiz — span **va** tur bir xilmi? **2-qadam.** Mos $\Rightarrow TP$; bashoratda bor, goldda yo'q $\Rightarrow FP$; goldda bor, bashoratda yo'q $\Rightarrow FN$.

Keltirib chiqarish: TP/FP/FN ni sanaymiz

1-qadam. Har bashorat entity ni haqiqiy (gold) entitylar bilan solishtiramiz — span **va** tur bir xilmi? **2-qadam.** Mos $\Rightarrow TP$; bashoratda bor, goldda yo'q $\Rightarrow FP$; goldda bor, bashoratda yo'q $\Rightarrow FN$. **3-qadam.** Bu sanoqlardan P, R, F_1 chiqadi:

$$P = \frac{TP}{TP+FP}, \quad R = \frac{TP}{TP+FN}, \quad F_1 = \frac{2PR}{P+R}$$

Xulosa: F1 — precision va recall ning harmonik o'rtachasi; ikkalasini ham muvozanatlaydi.

Qo'lda misol: entity F1 = 0.5

Gold: {(Malika, PER), (Toshkent, LOC)}; Bashorat: {(Malika, PER), (Toshkent, ORG)}.

Sanaymiz:

- (Malika, PER) — mos $\Rightarrow TP = 1$
- (Toshkent, ORG) — tur xato $\Rightarrow FP = 1$
- (Toshkent, LOC) — topilmadi $\Rightarrow FN = 1$

$$P = \frac{1}{2} = 0.5, R = \frac{1}{2} = 0.5$$

$$F_1 = \frac{2 \cdot 0.5 \cdot 0.5}{0.5 + 0.5} = \mathbf{0.5}$$

Toshkent chegarasi to'g'ri edi, ammo **tur** (LOC vs ORG) xato $\Rightarrow$ entity-darajada **to'liq xato**. Shuning uchun F1 atigi 0.5.

Sizning vazifangiz: hammasi to'g'ri bo'lsa

Savol — 60 soniya

Gold = Bashorat = $\{(Malika, PER), (Toshkent, LOC)\}$ (ikkalasi to'g'ri):

***TP, FP, FN* va F_1 qancha?**

Sizning vazifangiz: hammasi to'g'ri bo'lsa

Savol — 60 soniya

Gold = Bashorat = $\{(Malika, PER), (Toshkent, LOC)\}$ (ikkalasi to'g'ri):

TP , FP , FN va F_1 qancha?

Javob

$TP = 2, FP = 0, FN = 0 \Rightarrow P = 1.0, R = 1.0, F_1 = 1.0$.

Barcha entitylar (chegara + tur) to'g'ri $\Rightarrow$ mukammal F1. Real o'zbek NER da (kam ma'lumot) bunga erishish qiyin — [M].

Kod $\leftrightarrow$ formula: segeval bilan baholash

```
1 from segeval.metrics import f1_score,
2   classification_report
3
4 gold = [["B-PER", "B-LOC", "0", "0"]]
5 pred = [["B-PER", "B-ORG", "0", "0"]]
6
7 print(f1_score(gold, pred)) # 0.5
8 print(classification_report(gold, pred))
9 # entity-darajali P/R/F1 (token emas)
```

Kod $\rightarrow$ formula:

Kod	Formula
f1_score	F_1 (entity)
gold/pred	teg ketma-ketliklari

segeval IOB2 ni tushunadi va entity-darajali hisoblaydi. P10 da m10 ni shu bilan baholaymiz.

Baholash tuzog'i: token-darajali aniqlik chalg'itadi

Yuqori accuracy, yomon NER

Matnning $\sim 90\%$ i 0 bo'lishi mumkin. Hammani 0 deb teglagan model token-aniqligi ~ 0.9 — ammo **bironta entity** topmaydi ($F1 = 0$).

Yechim: har doim **entity-darajali** F1 (sequenceval) bilan baholang, token-accuracy bilan emas.

Prezentatsiya rejasi

- 1 Kirish: matndan obyektlarni ajratamiz
- 2 NER vazifasi va teglash sxemalari
- 3 LSTM asosidagi NER arxitekturasini
- 4 Bi-LSTM bilan kontekstni yaxshilash
- 5 NER samaradorligini baholash
- 6 Xulosa va amaliyotga ko'prik

O'zbek tilida: qo'shimchalar va kam ma'lumot NER ni qiynaydi

1. Joy nomi + qo'shimcha:

Toshkentda, Samarqanddan — joy nomi
"-da"/"-dan" bilan **bitta** token.

Xavf

Tokenizer "-da" ni ajratib yuborsa, entity chegarasi buziladi. IOB2 da Toshkentda butun token B-LOC bo'ladi.

2. Kam ma'lumot:

O'zbek NER uchun ochiq teglangan korpus kam (WikiANN ~200 jumlat).

Halol baho

F1 past bo'ladi (~40–60%) — buni **yashirmaymiz**. Bu o'zbek kabi kam-resursli tillar uchun real holat (P10 da ko'rsatamiz).

Sintez: teglash sxemalari va baholash

Sxema	Teglar	Izoh
IOB1	I-, B- (faqat ajratish uchun), O	B- kam ishlatiladi
IOB2	B-, I-, O	har entity B- bilan (kurs tanlovi)
BIOES	B-, I-, O, E-, S-	aniqroq, ammo ko'p teg

Asosiy g'oya

NER = ketma-ketlik teglash (IOB2) + Bi-LSTM (chap+o'ng kontekst) + entity-darajali F1 (segeval).
O'zbek uchun qo'shimcha va kam ma'lumot asosiy qiyinchiliklar.

Seminal manba: Lample va boshqalar (2016)

Lample, G., Ballesteros, M., Subramanian, S., Kawakami, K., Dyer, C. (2016). **Neural Architectures for Named Entity Recognition**. *NAACL-HLT 2016*.

Ahamiyati:

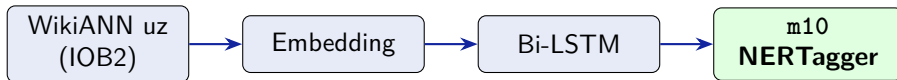
- Bi-LSTM-CRF NER arxitekturasini ommalashtirdi (lug'atsiz, kontekstdan).
- CRF qatlami teg ketma-ketligi cheklovlarini o'rganadi ([L1] muammosi).
- Ko'p tillarda (kam-resurslilar ham) standart bazaviy model.

Muhokama savoli (P10 gacha o'ylang)

O'zbek NER uchun teglangan ma'lumot kam (WikiANN ~200). Bu cheklovni qanday yengish mumkin? (Maslahat: transfer learning, ko'p tilli BERT — 4-hafta.)

- ① ✓ **Tushuntirdik:** NER vazifasi va IOB2 sxemasi (B-/I-/O).
- ② ✓ **Qo'lladik:** "Malika Toshkentda ishlaydi" $\rightarrow$ [B-PER, B-LOC, O, O].
- ③ ✓ **Tushuntirdik:** LSTM/Bi-LSTM NER arxitekturasini (har token uchun teg, $2 \times \text{hidden}$).
- ④ ✓ **Hisobladik:** entity-darajali $F1 = 0.5$ (tur xatosi).

Ertaga P10: Bi-LSTM bilan NER tizimini quramiz



P10 da qurasiz:

- m10 NERTagger: Embedding + Bi-LSTM + teg qatlami
- WikiANN uz da o'qitish, sequeval F1
- IOB2 teglashni tekshirish

Birinchi assert (bu slayd [I1]):

```
assert tags ==  
    ["B-PER", "B-LOC",  
     "O", "O"]
```

*# Ma'ruza L10 [I1]-slayd bilan
solishtiring (IOB2 teglash)*

- ❶ Lample, G., va boshq. (2016). *Neural Architectures for Named Entity Recognition*. NAACL-HLT 2016.
- ❷ Huang, Z., Xu, W., Yu, K. (2015). *Bidirectional LSTM-CRF Models for Sequence Tagging*. arXiv:1508.01991.
- ❸ Ramshaw, L., Marcus, M. (1995). *Text Chunking using Transformation-Based Learning*. (IOB sxemasi ildizi.)
- ❹ Pan, X., va boshq. (2017). *Cross-lingual NER (WikiANN)*. ACL 2017.
- ❺ seqeval hujjatlari — entity-darajali NER baholash.

Ilova S1: nega CRF qatlami qo'shiladi?

Per-token softmax teglarni **mustaqil** bashorat qiladi — imkonsiz o'tishlarni ($0 \rightarrow \text{I-LOC}$) to'sa olmaydi. **CRF** (Conditional Random Field) butun ketma-ketlik ehtimolini modellaydi:

$$P(\mathbf{y} \mid \mathbf{x}) \propto \exp \left(\sum_t [s(y_t, \mathbf{x}) + T(y_{t-1}, y_t)] \right)$$

bunda: s — emission balli (Bi-LSTM dan); $T(y_{t-1}, y_t)$ — teglar orasidagi o'tish balli (o'rganiladi); shu sababli imkonsiz o'tishlar past ball oladi.

Ilova S2: BIOES sxemasi haqida batafsil

IOB2 ga qo'shimcha ikki teg:

- E-XXX — entity **oxiri** (End).
- S-XXX — **yakka** tokenli entity (Single).

bunda: "Toshkent shahri" → B-LOC E-LOC; "Malika" → S-PER.

BIOES aniqroq chegara signali beradi (ba'zan F1 ni biroz oshiradi), ammo teglar soni ko'payadi ⇒ ko'proq ma'lumot talab qiladi. Kam-resursli o'zbek uchun IOB2 ko'pincha amaliyroq.